C343 Data Structures (Spring 2025) – Lab 8

Dr. A. Sabry

03/28/2025

The goal of this lab is to review Red-Black Trees with a few examples. Your major tasks:

1. Pull the code from `assignments/L8`.
2. Implement `rightRotateForLL` and `rightRotateForLR` in `RBTUtils.java`.
3. Implement `insertBalance` in `RedBlackTree.java`.
4. (Bonus) Implement `deleteBalance` in `RedBlackTree.java`.
5. Run the tests in `Main.java` and submit your code to your Github repo.

Review: Red-Black Trees

Despite the simplicity of AVL trees, they are not the preferred approach in production software compared to red-black trees. Both kinds of trees guarantee $O(\log(n))$ insertions, deletions, and search but red-black trees are often more efficient as they are less strict about the definition of balance. Technically, red-black trees maintain several invariants:

1. Every node is either red or black.
2. The root is black.
3. All empty nodes are black.
4. Red nodes cannot have red children.
5. Every path from a given node to any of the empty nodes contains the same number of black nodes.

These properties ensure that the longest path in a red-black tree is at most twice the length of the shortest path, maintaining logarithmic height. Indeed since the number of black nodes from the root to any empty node is guaranteed to be the same, the only difference in height must be due to the presence of red nodes. But the constraints ensure there will never be any two consecutive red nodes along any path.

Insertions/Deletions & Examples

Please refer to detailed explanations in `assignments/L8/notes.pdf`, Section 3.

Coding Tasks

There are two files containing TODOs under `assignments/L8/src`: `RBTUtils.java` and `RedBlackTree.java`.

In `RBTUtils.java`, we focus on 4 methods which correspond to 4 rotations (`leftRotateForRR`, `leftRotateForRL`, `rightRotateForLL`, `rightRotateForLR`). We have already implemented the first 2 of them for your reference.

In `RedBlackTree.java`, you need to implement `insertBalance` with the help of 4 rotation methods you just learnt in `RBTUtils.java` (for 2 bonus points, you may consider implement `deleteBalance` as well).

After you fill in all TODOs, please run the tests in `Main.java` to check correctness.